

3.1.3 Sets (Unordered, Unique, Math Operations)

سیٹ (Set) منفرد عناصر کا بغیر ترتیب اور بغیر انڈیکس مجموعہ ہے، جو {} میں لکھا جاتا ہے۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 04 of 18

3.1.3 Sets (Unordered, Unique, Math Operations)

LEARN IT
A **set** stores an **unordered, unindexed** collection of **unique** elements. It is inspired by mathematical sets. You write a set with **curly braces** `{}`. Duplicates are removed as soon as the set is created. Because there is no index, `myset[0]` is illegal. The set itself is **mutable** (you can add and remove items). Each item inside must be **immutable** (numbers, strings, tuples of immutables — not lists).

```
nums = {1, 4, 3, 3, 2, 2}
print(nums)
```

```
OUTPUT
{1, 2, 3, 4}
```

The two 3s and two 2s collapse to one each. Do not depend on the printed order: a set is unordered. CPython often prints small integers in a tidy way, but that is not a language promise.

Adding and removing

Method	Description
<code>.add(value)</code>	Adds one element.
<code>.update(values)</code>	Adds many elements from a list, tuple or set.
<code>.remove(value)</code>	Removes the value. Raises <code>KeyError</code> if it is missing.
<code>.discard(value)</code>	Removes the value. Does not crash if it is missing.
<code>.clear()</code>	Removes every element. Result is <code>set()</code> .

```
s = {1, 2, 3}
s.add(4)
print(s)
s.update([5, 6])
print(s)
s.discard(99)
print(s)
s.remove(1)
print(s)
s.clear()
print(s)
```

```
OUTPUT
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
{2, 3, 4, 5, 6}
set()
```

`discard(99)` quietly does nothing. After `clear()` Python prints `set()`, not `{}`, because `{}` already means an empty **dictionary**.

Compare with `.remove(99)` on a set that does not contain 99:

```
s = {1, 2, 3}
s.remove(99)
```

```
OUTPUT
Traceback (most recent call last):
  File "set_remove.py", line 2, in <module>
    s.remove(99)
  KeyError: 99
```

Prefer `.discard()` when the item might not be there, so a missing value does not crash the program.

Mathematical operations

Sets are built for union, intersection, difference and symmetric difference.



Overlap {3, 4} is the intersection. Left-only {1, 2} is $A - B$. Right-only {5, 6} is $B - A$.

Operation	Meaning	Syntax	Result for A, B above
Union	All unique items from both	$A \mid B$ or <code>A.union(B)</code>	<code>{1, 2, 3, 4, 5, 6}</code>
Intersection	Items in both	$A \& B$ or <code>A.intersection(B)</code>	<code>{3, 4}</code>
Difference	In A, not in B	$A - B$ or <code>A.difference(B)</code>	<code>{1, 2}</code>
Symmetric difference	In either, but not both	$A \wedge B$ or <code>A.symmetric_difference(B)</code>	<code>{1, 2, 5, 6}</code>

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)
print(A & B)
print(A - B)
print(A ^ B)
```

```
OUTPUT
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

$B - A$ would be `{5, 6}` — difference is not symmetric. Symmetric difference $A \wedge B$ is $(A - B) \mid (B - A)$.

Set traversal

There is no index, so you loop with `for`. Print order is not guaranteed. Use `sorted()` when you need a stable alphabetical or numerical order (it returns a list).

```
colors = ("red", "green", "blue")
for color in sorted(colors):
    print(color)
```

```
OUTPUT
blue
green
red
```

WORKED EXAMPLE

Deduplication. A teacher types roll numbers and some are repeated. A set keeps each roll once:

```
rolls = [12, 5, 12, 8, 5, 19]
unique = set(rolls)
print(unique)
print(sorted(unique))
```

```
OUTPUT
{8, 19, 12, 5}
[5, 8, 12, 19]
```

`set(rolls)` drops duplicates. `sorted(unique)` gives a list in numeric order for a register.

REMEMBER IT

SET = Same-Item Eaten Twice-never. Empty set is `set()`, never `{}`. Safe delete = `discard`; angry delete = `remove`.

COMMON MISTAKE

`myset[0]` raises `TypeError` because sets are unindexed. Convert with `list(myset)` only if you accept that the order is arbitrary. Do not put a list inside a set — lists are mutable, so they cannot be set elements.

آردو میں سمجھیں

سٹ (Set) منفرد عناصر کا بغیر ترتیب اور بغیر انڈیکس مجموعہ ہے، جو `{}` میں لکھا جاتا ہے۔ دہرائی ہوئی قیمتیں خود بخود ختم ہو جاتی ہیں۔ `add()` اور `update()` عناصر چوڑتے ہیں۔ `discard()` محفوظ طریقے سے ہٹاتا ہے جبکہ `remove()` غیر موجود عنصر پر `KeyError` دیتا ہے۔ اجتماع (Union) | اشتراک (Intersection) &، فرق - اور متوازی فرق ^ ریاضی کی کارروائیاں ہیں۔

CHECK YOURSELF

1. What is printed by `print({1, 4, 3, 3, 2, 2})`?

2. Why prefer `.discard()` over `.remove()`?

3. If $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$, what is $A \wedge B$?

Answers: 1. `{1, 2, 3, 4}` (duplicates removed; order may vary). 2. `discard` does not raise `KeyError` if the value is missing. 3. `{1, 2, 5, 6}`.

Next lecture: 3.1.4 Dictionaries (Key-Value Pairs)